

CS6868 Research Mini-Project

Software Transactional Memory (TL2)

Member One

ee23b137@smail.iitm.ac.in

Member Two

ee23b130@smail.iitm.ac.in

Member Three

ee23b184@smail.iitm.ac.in

May 1, 2026

Abstract

This project implements a Software Transactional Memory (STM) system based on the Transactional Locking II (TL2) algorithm in C++. The goal is to explore STM as an alternative to lock-based synchronization for building concurrent data structures.

We designed and implemented transactional versions of fundamental data structures which includes stack, queue, linked list, and a shared counter and evaluated their correctness and performance under concurrent workloads. To validate correctness, we employed dynamic analysis tools including ThreadSanitizer and AddressSanitizer, ensuring absence of data races and memory errors.

We evaluated throughput across varying thread counts and workloads, comparing our STM implementation against both lock-based equivalents and a KCAS-based implementation. Additionally, we analyzed performance bottlenecks using perf to identify optimization opportunities. We demonstrate STM composability by implementing an atomic bank transfer operation across multiple accounts.

Our results show that while STM provides a clean and composable abstraction, it incurs overhead due to validation and contention, limiting scalability at higher thread counts. Lock-based implementations often achieve higher throughput under low contention, while KCAS demonstrates competitive performance in select scenarios. Overall, the study highlights the trade-offs between correctness, composability, and performance in concurrent data structure design.

1 Goals

The primary goal of this project is to design, implement, and evaluate a Software Transactional Memory (STM) system based on the TL2 algorithm in C++. Using this STM framework, we implemented several concurrent data structures, including a shared counter, stack, bounded queue, and linked list. These data structures are designed to support safe concurrent access using transactional semantics rather than explicit locking.

This problem is interesting because concurrent programming is inherently difficult due to issues such as race conditions, deadlocks, and the complexity of fine-grained synchronization. Traditional lock-based approaches require careful reasoning about interleavings and often sacrifice composability. STM offers an alternative abstraction where operations are grouped into transactions that appear atomic, simplifying reasoning about correctness. However, this abstraction introduces overheads such as validation, versioning, and conflict detection, making its practical performance an important question.

This project connects directly to course topics on concurrent data structures, synchronization mechanisms, and non-blocking algorithms. In particular, it relates to lectures on Software Transactional Memory (e.g., TL2), compare-and-swap (CAS), and multi-word synchronization

primitives such as KCAS. The project also engages with broader themes of scalability, contention, and performance trade-offs in multicore systems.

To study these aspects, we compare our STM-based implementations against equivalent lock-based data structures and a KCAS-based approach under varying workloads and thread counts. We also design targeted tests to verify correctness under concurrent execution.

Research Question: How does a TL2-based STM implementation compare to lock-based and KCAS-based approaches in terms of correctness, scalability, and throughput across different workloads and levels of concurrency?

2 Background

2.1 Software Transactional Memory (STM)

Software Transactional Memory (STM) is a concurrency control mechanism that simplifies parallel programming by allowing threads to execute blocks of code as *transactions*. A transaction is a sequence of read and write operations that appears to execute atomically and in isolation, similar to transactions in databases [1].

Instead of using locks, STM systems allow optimistic execution: transactions proceed without blocking, while the system tracks conflicts and ensures correctness at commit time.

2.2 Core Concepts

Atomicity: A transaction either commits all its changes or aborts without modifying shared memory.

Isolation: Concurrent transactions do not observe intermediate states of each other.

Consistency: The system ensures memory correctness according to a defined consistency model (typically serializability).

Optimistic Concurrency: Transactions execute assuming no conflicts; validation occurs later.

2.3 Basic Execution Model

Each transaction maintains two data structures: a *read set*, which records all memory locations accessed during execution, and a *write set*, which stores updates locally without modifying shared memory. Writes are deferred and only applied during the commit phase, ensuring that intermediate states are not visible to other threads.

2.4 Transaction Lifecycle

1. **Begin:** Start a new transaction with a logical timestamp.
2. **Read:** Read values from memory while recording addresses in the read set.
3. **Write:** Store updates in the write set without modifying shared memory.
4. **Validation:** Ensure that no other transaction has modified the read set.
5. **Commit:** If validation succeeds, apply all writes.
6. **Abort:** If validation fails, discard changes and retry.

2.5 Example Pseudo-Code

```
procedure ATOMICALLY(transaction, arguments...)
    require: all arguments are copy-assignable

    saved_state ← copy(arguments)

    loop forever:
        START_TRANSACTION()

        result ← EXECUTE(transaction)

        if TRY_COMMIT() = SUCCESS then
            END_TRANSACTION()
            return result    // only if transaction returns a value

        else
            RESTORE(arguments, saved_state)
            // retry transaction
```

2.6 Conflict Detection and Versioning

TL2 [1] uses versioned locks together with a global logical clock. Each memory location is associated with a version number, and a transaction records the global version at its start. During validation, the transaction ensures that the versions of all previously read locations remain unchanged, thereby detecting conflicts.

2.7 Why STM is Useful

STM simplifies concurrent programming by eliminating deadlocks and allowing operations to be composed without explicit synchronization. However, these benefits come with overhead from bookkeeping and validation, and performance can degrade under high contention due to increased abort rates.

2.8 Relevance to This Project

In this project, we implement and evaluate an STM system inspired by TL2 [1], focusing on its performance under varying thread counts and workloads. We analyze throughput and contention behavior, connecting to course topics such as synchronization, memory consistency, and parallel scalability.

3 Tasks Undertaken

This section describes the design, implementation, and validation of our Software Transactional Memory (STM) system. We closely follow the TL2 algorithm [1], while making several engineering decisions to improve performance and robustness in a practical C++ implementation.

3.1 Implementation

We implemented a TL2-style STM system in C++, targeting correctness under concurrency and scalability across multiple threads. The implementation is available at:

<https://github.com/kuku929/tl2-cpp>

3.2 Directory Description

```
.  
|-- examples  
|-- graphs  
|-- perf  
|-- test  
|-- tl2  
'-- zoo
```

examples/ : Demonstrates STM composability via a concurrent bank transfer (atomic movement of funds between two accounts).

graphs/ : Contains benchmarking outputs and plotting scripts used to compare our STM against KCAS and mutex-based implementations.

perf/ : Performance evaluation harness and related code for measuring throughput and scalability under different workloads.

test/ : Correctness tests for STM-based data structures (e.g., stack, queue), executed with ThreadSanitizer and AddressSanitizer.

tl2/ : Implementation of the TL2 STM library, including core transactional mechanisms and metadata management.

zoo/ : Collection of concurrent data structures implemented using our C++ STM.

3.2.1 System Overview

Our STM system allows threads to execute critical sections as transactions without using locks explicitly. Instead, transactions execute optimistically and rely on validation and versioning to ensure correctness.

Each transaction maintains local metadata (read set and write set) and interacts with shared metadata (versioned locks and a global clock).

3.2.2 Key Data Structures

The read set stores (address, version) pairs for validation, while the write set buffers (address, value) updates until commit, ensuring atomic visibility and preventing exposure of intermediate state.

We implement two alternatives for maintaining these sets. The first uses an ordered structure, which maintains addresses in sorted order and simplifies deterministic lock acquisition during commit, at the cost of higher overhead and poorer cache locality. The second uses a hash-based index combined with contiguous storage, enabling faster lookups and better cache efficiency, but requiring explicit ordering before commit. The STM interface is designed to be extensible, allowing users to plug in custom data structures tailored to their workload.

Versioned Locks Each memory location is associated with a versioned lock that encodes both a version counter and a lock bit in a single atomic word, with the least significant bit representing the lock state. This compact representation allows synchronization and validation to be handled efficiently using atomic operations.

During reads, a transaction observes the version only if the location is not locked, ensuring a consistent snapshot. At commit time, locks are acquired using compare-and-swap, after which buffered writes are applied and version numbers are incremented before releasing the locks. Conflicts are detected by validating that previously read versions remain unchanged.

To improve scalability, locks are aligned to cache-line boundaries, avoiding false sharing and reducing coherence overhead under contention.

Global Clock A global monotonically increasing counter assigns begin timestamps and establishes a serialization order across transactions, ensuring consistency with the TL2 protocol.

3.2.3 Execution Flow

Each transaction follows the TL2 execution model:

1. **Begin:** The transaction reads the global clock to obtain its start timestamp.

2. **Read:** A read operation:

- Checks that the location is not locked
- Reads the value and version
- Verifies that the version is $\leq$ the transaction's start timestamp

The address and version are added to the read set.

3. **Write:** Writes are buffered in the write set. Subsequent reads to the same address are served from the write set (read-your-own-writes).

4. **Validation:** The transaction verifies that all entries in the read set still have the same version and are not locked.

5. **Commit:**

- Locks are acquired on all write-set locations using CAS
- The global clock is incremented to obtain a commit timestamp
- Validation is repeated to ensure no intervening writes occurred
- Buffered writes are applied to shared memory
- Locks are released with updated version numbers

6. **Abort:** If any step fails (validation or lock acquisition), the transaction aborts, releases any acquired locks, and retries.

3.3 Design Choices and Challenges

3.3.1 Generality of the STM Interface

The STM interface is designed to support arbitrary data types, including complex user-defined structures, as long as they provide correct copy and move semantics. Transactions operate on local snapshots created via copying, while move semantics are used where possible to reduce overhead. This design avoids specialized handling for different data types and enables the STM to remain general and reusable.

Pluggable Read/Write Set Design The STM decouples its core logic from the representation of read and write sets through an abstract interface. We implement two alternatives: an ordered set-based design that simplifies deterministic lock acquisition, and a hash-based design with contiguous storage that improves lookup performance and cache locality. This flexibility allows the choice of data structure to be guided by workload characteristics, and users can integrate custom implementations if needed.

Integration with `atomically` Transactions are expressed through an `atomically` interface that accepts a user-defined lambda. This preserves a sequential programming model while the STM transparently manages transaction execution, including retries on abort and restoration of modified arguments. The retry loop ensures that only successful executions become visible, providing atomicity without burdening the user with explicit synchronization.

Handling Heap-Allocated Data All updates are applied to transactional copies and become visible only after a successful commit. This prevents partial updates from being observed and ensures correctness even when working with heap-allocated or dynamically managed data structures. As a result, the STM maintains isolation without requiring additional user intervention.

3.3.2 Feature: Argument Restoration in `atomically`

Our STM implementation guarantees that modifications to lvalue arguments passed into a transaction are rolled back if the transaction aborts. In C++, lambdas may capture variables by reference, so updates inside an aborted transaction would otherwise persist, violating STM semantics.

To address this, the `atomically` interface takes lvalue arguments explicitly, snapshots their initial state before execution, and restores them on abort. This is integrated into the retry loop, ensuring each retry starts from a consistent state.

We validate this using a concurrent test where one transaction is forced to abort after partially modifying shared data and local arguments. After abort, all arguments are correctly restored, and no side effects leak outside the transaction. This ensures that transactional semantics are preserved even in the presence of language-level side effects.

3.3.3 Static Destruction Order Bug

We encountered a subtle *use-after-free* bug during program termination, identified using AddressSanitizer `:contentReference[oaicite:0]index=0`. The issue arose due to undefined destruction order of static objects in C++. Specifically, `TMan` was destroyed before other static objects whose destructors invoked `atomically`, leading to calls into an already freed transaction manager.

This resulted in invalid memory accesses when transaction state (e.g., logs) was accessed during late-stage destruction.

Resolution We resolved this by introducing an *unsafe, non-transactional access path* to `TVar` metadata. This allows destructors to directly read underlying values without invoking `atomically`, thereby avoiding reliance on `TMan` during static teardown.

Trade-off While this bypasses STM guarantees, it is safe in this context since execution is single-threaded during program termination.

3.3.4 Switch to single read/write set

To fix this we needed to switch to using a single set that. We need a single set to store the owner of a particular location. While locking the read-set we need to know if that location has already been locked by the same thread while it acquired the locks to its write-set. To maintain this information we need a common set.

3.4 Testing and Verification

3.4.1 Dynamic Analysis with Sanitizers

To ensure correctness and robustness, we used dynamic analysis tools including Thread Sanitizer (TSAN) and Address Sanitizer (ASAN). All concurrent workloads were executed with these sanitizers enabled.

No data races, synchronization errors, or memory safety violations (such as use-after-free or invalid accesses) were reported. This provides strong evidence that the implementation correctly enforces synchronization using atomic operations and maintains memory safety under concurrent execution.

3.4.2 Concurrent Workload Testing

We evaluated correctness using multiple concurrent data structures implemented with STM, including a shared counter, stack, queue, and linked list. These workloads were executed under varying thread counts to stress-test the system under different levels of contention.

Correctness was verified by ensuring that final states matched expected sequential outcomes, with no lost updates or inconsistent states observed across repeated runs.

All tests were executed with ThreadSanitizer (TSAN) and AddressSanitizer (ASAN) enabled. No data races, synchronization errors, or memory safety violations were reported, providing strong confidence in the correctness of the implementation.

High-contention scenarios were specifically used to induce frequent transaction aborts, validating the robustness of the retry logic and overall system stability.

4 Evaluation

NOTE: Due to the issue mentioned in 3.3.4 being spotted at the last minute, we had to make the switch to single set at the cost of performance. The plots and results in this section were created using an older version of our code than the final submission on GitHub. We hope to further improve the performance before the QnA session.

4.1 Experimental Setup

All experiments were conducted using a consistent benchmarking framework in C++ (compiled with `-O2`). We evaluated three implementations:

- Our TL2-based STM in C++
- Lock-based (mutex) implementations in C++
- STM-based implementations in OCaml using Kcas

Each data structure was subjected to identical workloads across implementations. Throughput was measured as operations per unit time, and experiments were repeated across varying thread counts:

$$\{1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$$

To reduce noise, we used sufficiently large workloads per thread and measured steady-state performance.

4.2 Workloads

We evaluated the following concurrent data structures:

Shared Counter Each thread performs a fixed number of increments on a shared counter.

- **Fine-grained:** one increment per transaction
- **Batched:** multiple increments (e.g., 1000) per transaction

Linked List We benchmarked the ordered set linked list implementation against C++ versions of the fine-grained, coarse-grained and lock-free implementations that were discussed in the course.

Stack and Queues For stack, bounded queue, and unbounded queue, each thread alternates between push and pop operations, generating mixed workloads with frequent conflicts.

4.3 Results

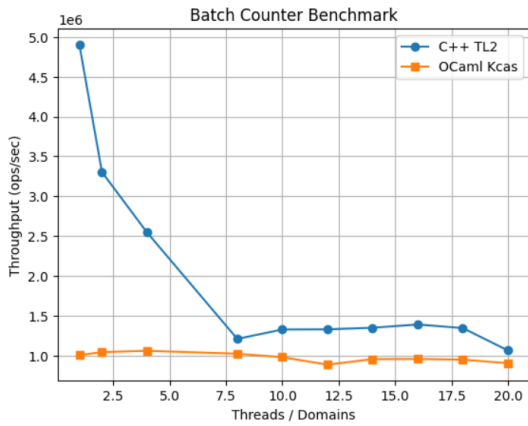


Figure 1: Throughput vs. threads (Batch Workload).

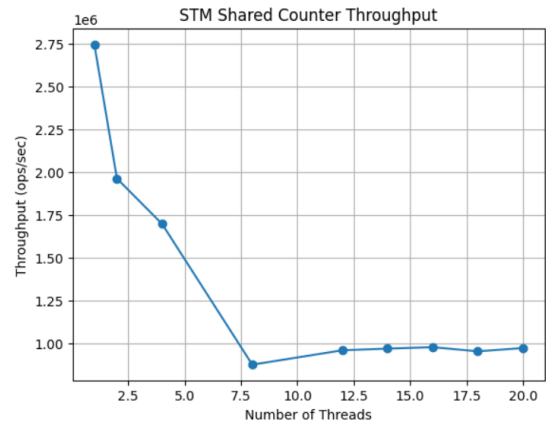


Figure 2: Throughput vs. threads (Fine grained Workload).

The shared counter highlights the effect of contention. In the fine-grained case, throughput saturates quickly due to frequent conflicts and retries. In contrast, batching significantly improves performance by amortizing transactional overhead and reducing contention frequency.

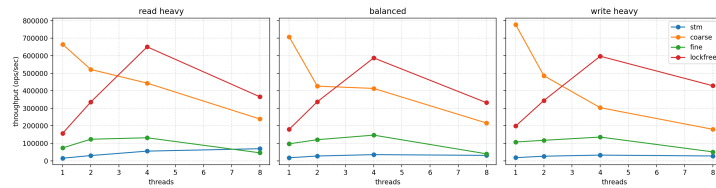


Figure 3: Throughput vs. threads for linked list.

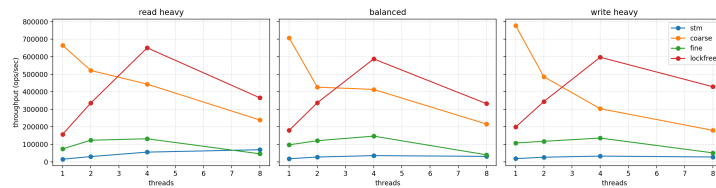


Figure 4: Throughput vs. threads for linked list.

The linked list workload shows moderate scalability. STM performs well during insertion phases but experiences contention during removals due to overlapping accesses. Compared to coarse-grained locking, STM benefits from finer-grained conflict detection but still incurs validation overhead.

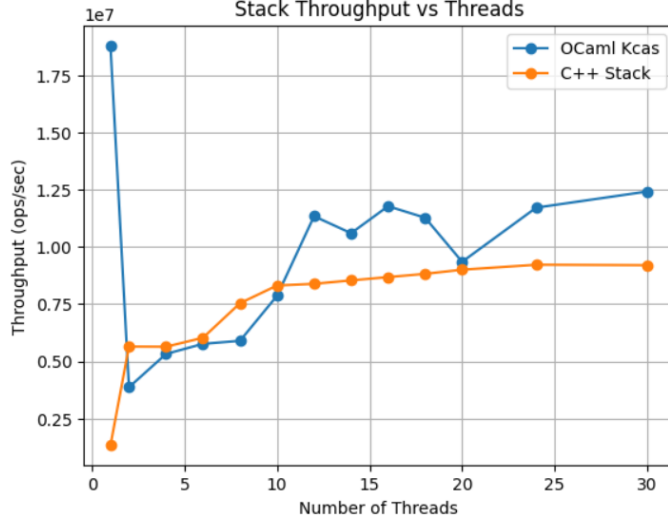


Figure 5: Throughput vs. threads for stack.

The stack workload demonstrates good scalability at lower thread counts. However, as threads increase, contention at the top of the stack limits scalability. STM overhead becomes more visible compared to lock-based approaches in this high-conflict scenario.

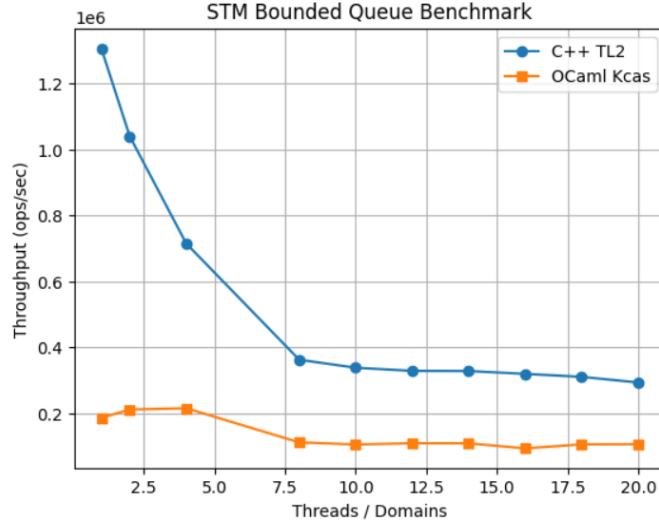


Figure 6: Throughput vs. threads for queues.

For queue workloads, STM scales reasonably well under moderate contention. The separation of enqueue and dequeue ends helps reduce conflicts, especially in the unbounded queue. However, under high thread counts, contention and retry overhead reduce throughput gains.

4.4 Discussion

Our STM implementation scales well under moderate contention and when transactions are sufficiently coarse-grained. Under these conditions, the overhead of validation and bookkeeping

is amortized, leading to improved throughput.

In contrast, performance degrades in highly contended and fine-grained workloads due to increased abort rates and repeated validation, which amplify synchronization costs and limit scalability.

Compared to mutex-based implementations, STM remains competitive while providing composability across operations. Relative to OCaml-based STM (Kcas), the C++ implementation achieves higher throughput, likely due to lower runtime overhead and finer control over memory and synchronization.

Overall, the results are consistent with expected STM behavior: performance is maximized when contention is controlled and transactional work is large enough to offset coordination overhead.

5 Reflection on the Use of LLMs

Tools and models used. We used ChatGPT (GPT-5.3) and GitHub Copilot. ChatGPT was primarily used for explaining the TL2 algorithm, clarifying STM concepts, debugging concurrency issues, and drafting parts of the report. Copilot was used for code completion and small helper functions during implementation.

What worked well. LLMs were effective in explaining the TL2 paper and suggesting a correct high-level structure for transactions (read/validate/write/commit). They also helped identify performance issues such as false sharing and suggested practical fixes (e.g., cache-line alignment), which improved implementation quality.

What did not work. LLMs were less reliable for low-level concurrency details. Some suggestions involving atomic operations and memory ordering were incomplete or incorrect, and a few explanations appeared plausible but did not hold under scrutiny. These issues were caught through testing, manual reasoning, and tools such as TSAN.

What was surprising. LLMs were unexpectedly strong at simplifying complex papers and accelerating initial understanding. However, they were less dependable for correctness-critical reasoning, particularly in edge cases involving concurrency.

What was difficult. LLMs could not replace reasoning about correctness under concurrency, including debugging subtle interleavings, validating retry logic, and ensuring proper synchronization and memory ordering. These required careful manual analysis and validation.

Your overall assessment. LLMs improved productivity and understanding but required careful verification for correctness. We would use them again in a similar role.

6 Conclusions

We set out to evaluate how a TL2-based STM compares with lock-based and KCAS-based approaches in terms of correctness, scalability, and throughput. Our results show that the TL2 STM implementation provides strong correctness guarantees and significantly improves composability, allowing complex operations (e.g., multi-object transactions) to be expressed cleanly without explicit synchronization.

In terms of performance, STM achieves competitive throughput under moderate contention and when transactions are sufficiently coarse-grained. Under these conditions, the overhead of validation and versioning is amortized, leading to reasonable scalability. However, in highly

contended or fine-grained workloads, STM performance degrades due to increased abort rates and repeated validation. In contrast, lock-based implementations often achieve higher throughput in low-contention scenarios due to lower overhead, while KCAS-based approaches offer a middle ground, with good performance in certain workloads but less general composability.

Overall, our findings highlight a key trade-off: STM improves programmability and composability at the cost of additional runtime overhead, which limits scalability under high contention.

Limitations Our evaluation is limited to a specific TL2 design and a fixed set of data structures and workloads. We did not explore advanced STM optimizations such as contention management policies, adaptive backoff, or hardware-assisted transactional memory. Additionally, comparisons with KCAS were made against an OCaml implementation, which may introduce language/runtime biases.

Future Work Given more time, we would explore optimization techniques such as better contention management, hybrid STM/locking approaches, and more efficient memory reclamation. Extending the evaluation to additional workloads and incorporating low-level optimizations (e.g., reducing validation overhead or improving cache locality further) would provide a more comprehensive understanding of STM performance.

Contributions

Member	Contribution (%)	Main responsibilities
Krutarth	34%	TL2 library implementation; unit testing; debugging; optimising
Dhruv	33%	Versioned locks, hash set (library) implementation, lock-based data structures, testing
Vaibhav	33%	STM-based data structures, KCAS vs STM benchmarking, plots, report
Total	100%	

References

- [1] Dave Dice, Ori Shalev, and Nir Shavit. Transactional locking ii. In *Proceedings of the 20th International Symposium on Distributed Computing (DISC)*, 2006. URL: https://people.csail.mit.edu/shanir/publications/Transactional_Locking.pdf.